

# Autoencoders, Regularisation, Bias–Variance and Beyond

Complete Lecture Notes with Full Derivations

---

*Course: Machine Learning / Deep Learning*  
*International Institute of Information Technology, Hyderabad*  
*Under the Guidance of Prof. Praveen Parchuri*

---

Autoencoders · PCA Equivalence · Loss Functions · Backpropagation · Regularisation (L2, Weight Tying, Denoising, Sparse, Contractive) · Bias-Variance Tradeoff · Training vs. Test Error · Stein's Lemma · Dataset Augmentation · Early Stopping · Ensemble Methods · Dropout · Discrete Cosine Transform · JPEG Compression

June 7, 2026

# Contents

---

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction to Autoencoders</b>                           | <b>3</b>  |
| 1.1      | What Is an Autoencoder? . . . . .                             | 3         |
| 1.2      | The Compression Perspective and Analogy with PCA . . . . .    | 3         |
| 1.3      | Under-Complete vs. Over-Complete Autoencoders . . . . .       | 4         |
| <b>2</b> | <b>Loss Functions for Autoencoders</b>                        | <b>4</b>  |
| 2.1      | Loss for Real-Valued Inputs: Mean Squared Error . . . . .     | 4         |
| 2.2      | Loss for Binary Inputs: Cross-Entropy . . . . .               | 4         |
| 2.3      | Derivation of Cross-Entropy from First Principles . . . . .   | 5         |
| 2.4      | Architecture Summary . . . . .                                | 5         |
| <b>3</b> | <b>Backpropagation Through an Autoencoder</b>                 | <b>5</b>  |
| 3.1      | Training Like a Regular Network . . . . .                     | 5         |
| 3.2      | Key Gradient: Binary Cross-Entropy + Sigmoid Output . . . . . | 5         |
| <b>4</b> | <b>Link Between PCA and Linear Autoencoders</b>               | <b>6</b>  |
| 4.1      | Conditions for PCA–AE Equivalence . . . . .                   | 6         |
| 4.2      | The Optimisation Problem . . . . .                            | 6         |
| 4.3      | Connection to SVD and the Eckart–Young Theorem . . . . .      | 6         |
| 4.4      | Full Derivation of the Encoder $W$ . . . . .                  | 6         |
| <b>5</b> | <b>Regularisation on Autoencoders</b>                         | <b>7</b>  |
| 5.1      | Why Regularisation Is Needed . . . . .                        | 7         |
| 5.2      | L2 Regularisation (Weight Decay) . . . . .                    | 7         |
| 5.2.1    | Geometric Interpretation via Taylor Expansion . . . . .       | 7         |
| 5.2.2    | Spectral Analysis of L2 Regularisation . . . . .              | 8         |
| 5.3      | Weight Tying . . . . .                                        | 8         |
| <b>6</b> | <b>Denoising Autoencoders</b>                                 | <b>8</b>  |
| 6.1      | Motivation and Core Idea . . . . .                            | 8         |
| 6.2      | Corruption Processes . . . . .                                | 8         |
| 6.3      | Visualising Hidden Neurons . . . . .                          | 9         |
| 6.3.1    | Visual Comparison: Vanilla AE vs. Denoising AE . . . . .      | 9         |
| <b>7</b> | <b>Sparse Autoencoders</b>                                    | <b>9</b>  |
| 7.1      | Motivation: Sparsity Constraint . . . . .                     | 9         |
| 7.2      | KL Divergence Penalty . . . . .                               | 9         |
| 7.3      | Gradient of the Sparsity Term . . . . .                       | 10        |
| <b>8</b> | <b>Contractive Autoencoders</b>                               | <b>10</b> |
| 8.1      | Regularisation Methods Comparison . . . . .                   | 10        |
| <b>9</b> | <b>Bias, Variance and Model Complexity</b>                    | <b>10</b> |
| 9.1      | Experimental Setup . . . . .                                  | 10        |
| 9.2      | Formal Definitions . . . . .                                  | 11        |
| 9.3      | Bias–Variance Decomposition . . . . .                         | 11        |

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| <b>10 Training Error vs. Test Error</b>                                 | <b>11</b> |
| 10.1 Why Test Error Approximates True Error . . . . .                   | 11        |
| <b>11 True Error and Model Complexity: Stein's Lemma</b>                | <b>12</b> |
| <b>12 L2 Regularisation in Depth</b>                                    | <b>12</b> |
| <b>13 Dataset Augmentation</b>                                          | <b>12</b> |
| <b>14 Parameter Sharing and Tying</b>                                   | <b>13</b> |
| <b>15 Adding Noise to Inputs</b>                                        | <b>13</b> |
| <b>16 Adding Noise to Outputs (Label Smoothing)</b>                     | <b>13</b> |
| <b>17 Early Stopping</b>                                                | <b>13</b> |
| <b>18 Ensemble Methods and Bagging</b>                                  | <b>14</b> |
| 18.1 Bagging MSE Analysis . . . . .                                     | 14        |
| <b>19 Dropout</b>                                                       | <b>14</b> |
| 19.1 The Dropout Mechanism . . . . .                                    | 14        |
| 19.2 Why Dropout Works: Preventing Co-Adaptation . . . . .              | 15        |
| 19.3 Test Time: Weight Scaling . . . . .                                | 15        |
| <b>20 Discrete Cosine Transform (DCT) and JPEG Compression</b>          | <b>15</b> |
| 20.1 2D DCT Formula . . . . .                                           | 15        |
| 20.2 JPEG Compression Pipeline . . . . .                                | 15        |
| <b>21 Complete Summary</b>                                              | <b>16</b> |
| 21.1 Key Equations at a Glance . . . . .                                | 16        |
| 21.2 Regularisation Methods Summary . . . . .                           | 17        |
| <b>A Appendix A: Noise Inputs = L2 Regularisation (Full Derivation)</b> | <b>17</b> |
| <b>B Appendix B: Sparse AE – KL Divergence Gradient</b>                 | <b>17</b> |
| <b>C Appendix C: Contractive AE – Jacobian Norm for Sigmoid Units</b>   | <b>18</b> |
| <b>D Appendix D: Early Stopping – Full L2 Correspondence</b>            | <b>18</b> |
| <b>E Appendix E: Bagging MSE – Full Derivation</b>                      | <b>18</b> |

# 1 Introduction to Autoencoders

## 1.1 What Is an Autoencoder?

An autoencoder is a special type of feed-forward neural network that performs two operations: it **encodes** input data into a compact hidden representation, then **decodes** that representation back to reconstruct the original input. The architecture can be written as:

$$\mathbf{x}_i \xrightarrow{W, \mathbf{b}} \mathbf{h} \xrightarrow{W^*, \mathbf{c}} \hat{\mathbf{x}}_i$$

### Definition

**Encoder:** Given input  $\mathbf{x}_i \in \mathbb{R}^n$ , the encoder maps it to a hidden (latent) representation:

$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b}) \in \mathbb{R}^d$$

**Decoder:** The decoder maps the hidden representation back to a reconstruction:

$$\hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c}) \in \mathbb{R}^n$$

where  $g(\cdot)$  is the encoder nonlinearity,  $f(\cdot)$  is the decoder nonlinearity,  $W \in \mathbb{R}^{d \times n}$  are encoder weights,  $W^* \in \mathbb{R}^{n \times d}$  are decoder weights, and  $\mathbf{b} \in \mathbb{R}^d$ ,  $\mathbf{c} \in \mathbb{R}^n$  are biases.

### Notation Summary:

- $\mathbf{x}_i \in \mathbb{R}^n$ : the  $i$ -th input data point; data matrix  $X \in \mathbb{R}^{m \times n}$  has  $m$  samples and  $n$  features
- $\mathbf{h} \in \mathbb{R}^d$ : hidden / latent representation
- $\hat{\mathbf{x}}_i \in \mathbb{R}^n$ : reconstructed output
- $g(\cdot)$ : encoder nonlinearity (sigmoid, tanh, ReLU)
- $f(\cdot)$ : decoder nonlinearity (depends on input type)

### Key Insight

**Goal:** Learn the most important characteristics of  $\mathbf{x}_i$  so that  $\mathbf{h}$  captures everything needed to reconstruct the original input. The model is trained to minimise a reconstruction loss ensuring  $\hat{\mathbf{x}}_i$  is close to  $\mathbf{x}_i$ .

## 1.2 The Compression Perspective and Analogy with PCA

Consider the case  $d < n$  (hidden dimension smaller than input dimension). If we can still reconstruct  $\hat{\mathbf{x}}_i$  from  $\mathbf{h}$ , then  $\mathbf{h}$  is a **loss-free (or low-loss) encoding**.

### Example

Think of it like file compression: 10 bits  $\rightarrow$  4 bits  $\rightarrow$  10 bits. If the decoded file equals the original, the 4-bit representation captured everything essential.

### Analogy with PCA:

- **PCA:** Projects data onto the top- $k$  principal components, discarding noise, low-variance, and correlated dimensions.
- **Autoencoder hidden layer  $h$ :** Performs similar dimensionality reduction, but *non-linearly*.
- Both seek a **compact representation** of the data.

### 1.3 Under-Complete vs. Over-Complete Autoencoders

#### Definition

- **Under-complete autoencoder:**  $d < n$ . Forces the network to learn a concise summary.
- **Over-complete autoencoder:**  $d \geq n$ . The network *could* learn a trivial identity encoding.

#### Important

**Key Problem with Over-Complete AEs:** Without additional constraints, an over-complete autoencoder has no incentive to learn a meaningful representation – it will simply memorise (copy) training data (**identity encoding**), which is useless in practice. This motivates **regularisation**.

## 2 Loss Functions for Autoencoders

### 2.1 Loss for Real-Valued Inputs: Mean Squared Error

When inputs  $\mathbf{x}_i \in \mathbb{R}^n$  are real-valued, the natural loss is the **mean squared error (MSE)**:

$$\mathcal{L}(\theta) = \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n (\hat{x}_{ij} - x_{ij})^2 = \frac{1}{m} \sum_{i=1}^m (\mathbf{x}_i - \hat{\mathbf{x}}_i)^\top (\hat{\mathbf{x}}_i - \mathbf{x}_i)$$

**Decoder choice:** Most appropriate decoder is **linear**, i.e.  $\hat{\mathbf{x}}_i = W^* \mathbf{h} + \mathbf{c} \in \mathbb{R}^n$ . The encoder  $g$  is typically chosen as the **sigmoid** function.

### 2.2 Loss for Binary Inputs: Cross-Entropy

When inputs  $x_{ij} \in \{0, 1\}$  are binary, use a **sigmoid (logistic)** decoder:

$$\hat{x}_{ij} = \sigma(a_j) = \frac{1}{1 + e^{-a_j}}$$

Output is interpreted as the probability that bit  $j$  is 1. The **cross-entropy loss** is:

$$\mathcal{L}(\theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n \left[ x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij}) \right]$$

## 2.3 Derivation of Cross-Entropy from First Principles

Let  $p = [x_{ij}, 1 - x_{ij}]^\top$  be the true binary distribution and  $q = [\hat{x}_{ij}, 1 - \hat{x}_{ij}]^\top$  be the estimated distribution. The cross-entropy between  $p$  and  $q$ :

$$H(p, q) = - \sum_{k=0}^1 p_k \log q_k = - [x_{ij} \log \hat{x}_{ij} + (1 - x_{ij}) \log(1 - \hat{x}_{ij})]$$

This is minimised when  $\hat{x}_{ij} = x_{ij}$ .

### Key Insight

#### Why Cross-Entropy $\succ$ Squared Loss for Binary inputs?

Cross-entropy naturally aligns with the Bernoulli probabilistic model. It avoids saturation in gradient computation (the sigmoid gradient cancels cleanly) and works much better in practice for binary outputs.

## 2.4 Architecture Summary

|             | Binary Inputs | Real-Valued Inputs                       |
|-------------|---------------|------------------------------------------|
| Encoder $g$ | Sigmoid       | Sigmoid or tanh                          |
| Decoder $f$ | Sigmoid       | Linear ( $W^* \mathbf{h} + \mathbf{c}$ ) |
| Loss        | Cross-entropy | Mean Squared Error                       |

## 3 Backpropagation Through an Autoencoder

### 3.1 Training Like a Regular Network

An autoencoder is trained exactly like a feed-forward neural network using **backpropagation**. We need:

$$\frac{\partial \mathcal{L}(\theta)}{\partial W^*}, \quad \frac{\partial \mathcal{L}(\theta)}{\partial W}, \quad \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{c}}, \quad \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{b}}$$

The weight  $W$  appears in both encoder and decoder paths, so:

$$\frac{\partial \mathcal{L}}{\partial W} = \left( \frac{\partial \mathcal{L}}{\partial W} \right)_{\text{encoder}} + \left( \frac{\partial \mathcal{L}}{\partial W} \right)_{\text{decoder}}$$

### 3.2 Key Gradient: Binary Cross-Entropy + Sigmoid Output

With  $h_{2j} = \hat{x}_{ij} = \sigma(a_{2j})$  and cross-entropy loss, a remarkable simplification occurs:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial a_{2j}} &= \left( -\frac{x_{ij}}{\hat{x}_{ij}} + \frac{1 - x_{ij}}{1 - \hat{x}_{ij}} \right) \cdot \hat{x}_{ij}(1 - \hat{x}_{ij}) \\ &= -x_{ij}(1 - \hat{x}_{ij}) + (1 - x_{ij})\hat{x}_{ij} \\ &= \hat{x}_{ij} - x_{ij} \end{aligned}$$

**Key Insight**

The gradient is simply the prediction error  $\hat{x}_{ij} - x_{ij}$ . This clean result is why sigmoid + cross-entropy is such a natural pairing for binary outputs.

## 4 Link Between PCA and Linear Autoencoders

### 4.1 Conditions for PCA–AE Equivalence

**Theorem / Key Result**

The encoder of a linear autoencoder is **equivalent to PCA** when all four conditions hold:

1. The encoder is **linear**:  $\mathbf{h} = W\mathbf{x}$  (both  $g$  and  $f$  are linear)
2. The decoder is **linear**:  $\hat{\mathbf{x}} = W^*\mathbf{h} + \mathbf{c}$
3. The loss function is **mean squared error**
4. Inputs are **normalised** to zero mean:

$$\hat{x}_{ij} = \frac{1}{\sqrt{m}} \left( x_{ij} - \frac{1}{m} \sum_{k=1}^m x_{kj} \right)$$

After normalisation,  $X^\top X$  equals the **covariance matrix**, which plays a central role in PCA.

### 4.2 The Optimisation Problem

With linear encoder and decoder, the autoencoder objective becomes:

$$\min_{W, W^*} \|X - HW^*\|_F^2$$

where  $X \in \mathbb{R}^{m \times n}$ ,  $H = XW^\top \in \mathbb{R}^{m \times k}$ ,  $\hat{X} = HW^* \in \mathbb{R}^{m \times n}$ , and  $\|A\|_F = \sqrt{\sum_{ij} a_{ij}^2}$  is the **Frobenius norm**.

### 4.3 Connection to SVD and the Eckart–Young Theorem

**Eckart–Young–Mirsky Theorem**

The best rank- $k$  approximation to  $X$  in Frobenius norm is:

$$\hat{X} = U_{:, \leq k} \Sigma_{k \times k} V_{:, \leq k}^\top$$

where  $X = U\Sigma V^\top$  is the full SVD of  $X$ .

By matching variables in  $HW^* = U_{:, \leq k} \Sigma_{k \times k} V_{:, \leq k}^\top$ , one solution is  $H = U_{:, \leq k} \Sigma_{k \times k}$ ,  $W^* = V_{:, \leq k}^\top$ .

### 4.4 Full Derivation of the Encoder $W$

Starting from  $H = U_{:, \leq k} \Sigma_{k \times k}$  and pre-multiplying by  $(XX^\top)(XX^\top)^{-1} = I$ :

Using  $X = U\Sigma V^\top$ :

$$XX^\top = U\Sigma^2U^\top, \quad XV = U\Sigma$$

After careful substitution and using  $V^\top V = I$ ,  $U^\top U = I$ :

$$H = XV\Sigma^{-1}I_{\cdot, \leq k} \Sigma_{k \times k} = XV_{\cdot, \leq k}$$

### PCA–AE Equivalence (Result)

The optimal linear autoencoder gives:

$$W = V_{\cdot, \leq k}^\top, \quad H = XV_{\cdot, \leq k}, \quad W^* = V_{\cdot, \leq k}^\top$$

Since  $V$  = right singular vectors of  $X$  = eigenvectors of  $X^\top X$  = eigenvectors of the covariance matrix (after normalisation), the encoder matrix  $W$  and the PCA projection matrix  $P$  are identical. **QED**

## 5 Regularisation on Autoencoders

### 5.1 Why Regularisation Is Needed

#### Important

**Key principle:** Any kind of regularisation always tries to **restrict the freedom of parameters**. Over-complete AEs can trivially copy  $\mathbf{x}_i \rightarrow \mathbf{h} \rightarrow \hat{\mathbf{x}}_i$  (identity encoding). Even under-complete AEs with high-dimensional inputs can overfit. This motivates regularisation.

### 5.2 L2 Regularisation (Weight Decay)

$$\min_{\theta} \underbrace{\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n (\hat{x}_{ij} - x_{ij})^2}_{\text{reconstruction loss}} + \underbrace{\lambda \|\theta\|^2}_{\text{L2 penalty}}$$

For  $\tilde{\mathcal{L}}(w) = \mathcal{L}(w) + \frac{\alpha}{2} \|w\|^2$ , the SGD update becomes:

$$w_{t+1} = (1 - \eta\alpha)w_t - \eta\nabla\mathcal{L}(w_t)$$

The factor  $(1 - \eta\alpha)$  gradually shrinks weights – hence the name **weight decay**.

#### 5.2.1 Geometric Interpretation via Taylor Expansion

Let  $w^*$  be the unregularised optimal solution. Using Taylor expansion to second order:

$$\nabla\mathcal{L}(w) \approx H(w - w^*)$$

where  $H$  is the Hessian. Setting the regularised gradient to zero at  $\tilde{w}$ :

$$(H + \alpha I)\tilde{w} = Hw^* \implies \tilde{w} = (H + \alpha I)^{-1}Hw^*$$



### 5.2.2 Spectral Analysis of L2 Regularisation

Using spectral decomposition  $H = Q\Lambda Q^\top$ :

$$\tilde{w} = Q D Q^\top w^*, \quad D_{ii} = \frac{\lambda_i}{\lambda_i + \alpha}$$

#### Key Insight

**Only significant directions (larger eigenvalues) are retained.**

- If  $\lambda_i \gg \alpha$ : scaling factor  $\approx 1$  (direction retained)
- If  $\lambda_i \ll \alpha$ : scaling factor  $\approx 0$  (direction shrunk to zero)

**Effective number of parameters**  $= \sum_{i=1}^n \frac{\lambda_i}{\lambda_i + \alpha}$

## 5.3 Weight Tying

#### Definition

**Weight tying** enforces  $W^* = W^\top$ : the decoder weight matrix is set to the transpose of the encoder weight matrix. This reduces free parameters from  $2d \times n$  down to  $d \times n$  (exactly half), reducing overfitting.

## 6 Denoising Autoencoders

### 6.1 Motivation and Core Idea

A **denoising autoencoder** (DAE) corrupts the input before feeding it to the network, but still tries to reconstruct the *original* uncorrupted input.

**Corruption process:** A probabilistic process  $P(\tilde{x}_{ij} \mid x_{ij})$  generates a corrupted version  $\tilde{\mathbf{x}}_i$ .

**Objective:** Minimise  $\arg \min_{\theta} \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n (\hat{x}_{ij} - x_{ij})^2$  while feeding  $\tilde{\mathbf{x}}_i$  to the network.

### 6.2 Corruption Processes

**Type 1 – Masking Noise:**

$$P(\tilde{x}_{ij} = 0 \mid x_{ij}) = q, \quad P(\tilde{x}_{ij} = x_{ij} \mid x_{ij}) = 1 - q$$

**Type 2 – Gaussian Noise:**

$$\tilde{x}_{ij} = x_{ij} + \mathcal{N}(0, \sigma^2)$$

#### Key Insight

With corruption, it no longer makes sense for the model to copy  $\tilde{\mathbf{x}}_i$  into  $\mathbf{h}$  and then into  $\hat{\mathbf{x}}_i$  – the objective will *not* be minimised. The model must learn to reconstruct

corrupted features by relying on **interactions with other features**, thus capturing true data structure.

### 6.3 Visualising Hidden Neurons

For neuron  $\ell$  with  $h_\ell = \sigma(w_\ell^\top \mathbf{x}_i)$ , find the maximally activating input:

$$\max_{\mathbf{x}_i} w_\ell^\top \mathbf{x}_i \quad \text{s.t.} \quad \|\mathbf{x}_i\|^2 = 1$$

By the Cauchy-Schwarz inequality, the maximum is achieved when  $\mathbf{x}_i \propto w_\ell$ :

$$\mathbf{x}_i = \frac{w_\ell}{\|w_\ell\|}$$

The normalised weight vector of each neuron is its maximally activating input.

#### 6.3.1 Visual Comparison: Vanilla AE vs. Denoising AE

- **Vanilla AE** (no noise): Neurons look noisy or random; do not learn meaningful patterns.
- **Denoising AE with  $q = 0.25$** : Neurons resemble pen-stroke detectors; more structured.
- **Denoising AE with  $q = 0.5$** : Filters become wider – as noise increases, the neuron relies on *adjacent pixels*, so filters become broader (larger receptive field).

## 7 Sparse Autoencoders

### 7.1 Motivation: Sparsity Constraint

#### Definition

A **sparse autoencoder** ensures each hidden neuron is **inactive most of the time**. The **average activation** of neuron  $\ell$  over the dataset:

$$\hat{\rho}_\ell = \frac{1}{m} \sum_{i=1}^m h(\mathbf{x}_i)_\ell$$

A sparse autoencoder uses sparsity parameter  $\rho \approx 0.005$  and enforces  $\hat{\rho}_\ell = \rho$ .

### 7.2 KL Divergence Penalty

To enforce  $\hat{\rho}_\ell = \rho$ , add a sum of KL divergences to the objective:

$$\mathcal{S}(\theta) = \sum_{\ell=1}^k \left[ \rho \log \frac{\rho}{\hat{\rho}_\ell} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_\ell} \right] = \sum_{\ell=1}^k \text{KL}(\text{Bern}(\rho) \parallel \text{Bern}(\hat{\rho}_\ell)) \geq 0$$

with equality iff  $\hat{\rho}_\ell = \rho$  for all  $\ell$ . The full loss:  $\tilde{\mathcal{L}}(\theta) = \mathcal{L}(\theta) + \mathcal{S}(\theta)$ .

### 7.3 Gradient of the Sparsity Term

$$\frac{\partial \mathcal{S}}{\partial \hat{\rho}_\ell} = -\frac{\rho}{\hat{\rho}_\ell} + \frac{1-\rho}{1-\hat{\rho}_\ell}, \quad \frac{\partial \hat{\rho}_\ell}{\partial w} = \mathbf{x}_i (g'(w^\top \mathbf{x}_i + \mathbf{b}))^\top$$

$$\frac{\partial \tilde{\mathcal{L}}(\theta)}{\partial w} = \frac{\partial \mathcal{L}(\theta)}{\partial w} + \frac{\partial \mathcal{S}(\theta)}{\partial w}$$

## 8 Contractive Autoencoders

A **contractive autoencoder** (CAE) penalises the **sensitivity** of the hidden representation to changes in the input:

$$\mathcal{S}(\theta) = \|J_f(\mathbf{h})\|_F^2 = \sum_{j=1}^n \sum_{\ell=1}^k \left( \frac{\partial h_\ell}{\partial x_j} \right)^2$$

#### Key Insight

**The Beautiful Contradiction:**

- $\mathcal{L}(\theta)$ : wants  $\mathbf{h}$  to capture *all* variations in  $\mathbf{x}$  (sensitivity needed for reconstruction)
- $\mathcal{S}(\theta)$ : penalises *any* sensitivity to  $\mathbf{x}$
- **Result:**  $\mathbf{h}$  is sensitive only to the most important (high-variance) variations in the data – analogous to PCA.

For sigmoid hidden units  $h_\ell = \sigma(w_\ell^\top \mathbf{x} + b_\ell)$ :

$$\|J_f(\mathbf{h})\|_F^2 = \sum_{\ell=1}^k (h_\ell(1-h_\ell))^2 \|w_\ell\|^2$$

### 8.1 Regularisation Methods Comparison

| Method            | Regularisation Term                           | What It Controls          |
|-------------------|-----------------------------------------------|---------------------------|
| L2 / Weight Decay | $\lambda \ \theta\ ^2$                        | Weight magnitude          |
| Sparse AE (KL)    | $\sum_\ell \text{KL}(\rho \ \hat{\rho}_\ell)$ | Average activation        |
| Contractive AE    | $\ J_f(\mathbf{h})\ _F^2$                     | Sensitivity to input      |
| Denoising AE      | Corruption $P(\tilde{x}   x)$                 | Implicit (via corruption) |
| Weight Tying      | $W^* = W^\top$                                | Parameter count           |

## 9 Bias, Variance and Model Complexity

### 9.1 Experimental Setup

Consider fitting a sinusoidal function with two models:

- **Simple model:**  $y = w_1 x + w_0$  (degree 1)
- **Complex model:**  $y = \sum_{i=1}^{25} w_i x^i + w_0$  (degree 25)

- **Simple models:** Do not differ much across samples (low variance) but are far from the true curve.  $\Rightarrow$  **High bias, low variance (underfitting)**
- **Complex models:** Very different from each other across samples (high variance).  $\Rightarrow$  **Low bias, high variance (overfitting)**

## 9.2 Formal Definitions

### Definition

$$\text{Bias}(\hat{f}(x)) = \mathbb{E}[\hat{f}(x)] - f(x) \quad \text{Var}(\hat{f}(x)) = \mathbb{E}\left[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2\right]$$

## 9.3 Bias–Variance Decomposition

### Bias–Variance–Noise Decomposition

For any data point  $(x, y)$  with  $y = f(x) + \varepsilon$ ,  $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ :

$$\mathbb{E}\left[(y - \hat{f}(x))^2\right] = \underbrace{(\text{Bias}(\hat{f}(x)))^2}_{\text{systematic error}} + \underbrace{\text{Var}(\hat{f}(x))}_{\text{sensitivity to training set}} + \underbrace{\sigma^2}_{\text{irreducible noise}}$$

**Proof sketch:** Define  $A = f - \mathbb{E}[\hat{f}]$ ,  $B = \varepsilon$ ,  $C = \mathbb{E}[\hat{f}] - \hat{f}$ . Then  $y - \hat{f} = A + B + C$ , and all cross terms  $\mathbb{E}[AB]$ ,  $\mathbb{E}[BC]$ ,  $\mathbb{E}[AC]$  vanish because  $\varepsilon$  has zero mean and is independent of model randomness, while  $A$  is a constant w.r.t. model randomness. The remaining diagonal terms give the three components. **QED**

## 10 Training Error vs. Test Error

### Definition

- **Training error:**  $\frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2$
- **Test error:**  $\frac{1}{n_{\text{test}}} \sum_{i=n+1}^{n+n_{\text{test}}} (y_i - \hat{f}(x_i))^2$

### 10.1 Why Test Error Approximates True Error

For **test observations** (not used in training),  $\varepsilon_i \perp \hat{f}(x_i)$ , so:

$$\mathbb{E}[\varepsilon_i(\hat{f}(x_i) - f(x_i))] = \mathbb{E}[\varepsilon_i] \cdot \mathbb{E}[\hat{f}(x_i) - f(x_i)] = 0$$

$$\mathbb{E}[(\hat{f}(x_i) - f(x_i))^2] \approx \text{Empirical test error} + \text{small constant}$$

For **training observations**,  $\varepsilon_i$  was used to estimate parameters, so  $\text{Cov}(\varepsilon_i, \hat{f}(x_i) - f(x_i)) \neq 0$  and training error is **optimistically biased downward**.

**Important**

Always use a validation set (independent of training set) to estimate error.

## 11 True Error and Model Complexity: Stein's Lemma

**Stein's Lemma**

If  $X \sim \mathcal{N}(\mu, \sigma^2)$  and  $g$  is differentiable with  $\mathbb{E}[|g'(X)|] < \infty$ , then:

$$\mathbb{E}[(X - \mu)g(X)] = \sigma^2 \mathbb{E}[g'(X)]$$

**Application:** Using Stein's Lemma (with some technical steps):

$$\frac{1}{n} \sum_{i=1}^n \varepsilon_i(\hat{f}(x_i) - f(x_i)) = \frac{\sigma^2}{n} \sum_{i=1}^n \frac{\partial \hat{f}(x_i)}{\partial y_i}$$

$$\text{True error} = \text{Empirical train error} + \underbrace{\frac{2\sigma^2}{n} \sum_{i=1}^n \frac{\partial \hat{f}(x_i)}{\partial y_i}}_{\mathcal{S}(\text{model complexity})}$$

**Key Insight**

This is the **mathematical foundation for all regularisation methods**. During training, minimise:

$$\min_{\theta} [\mathcal{L}_{\text{train}}(\theta) + \mathcal{S}(\theta)]$$

where  $\mathcal{S}(\theta)$  is high for complex models and small for simple models. L2, weight tying, dataset augmentation, denoising, early stopping, ensemble methods, and dropout are all instances of this framework.

## 12 L2 Regularisation in Depth

$$\tilde{\mathcal{L}}(w) = \mathcal{L}(w) + \frac{\alpha}{2} \|w\|^2$$

SGD update with L2:

$$w_{t+1} = (1 - \eta\alpha)w_t - \eta\nabla\mathcal{L}(w_t)$$

**Geometric intuition:** L2 regularisation restricts the model to a ball in weight space, preventing weights from growing large and thus limiting overfitting on training data.

## 13 Dataset Augmentation

Certain transformations to an input do not change its label. We exploit this to create additional training examples:

- Rotation (e.g. by  $5^\circ$ ), horizontal/vertical shift, blurring, horizontal flip, changing some pixels

**Why it helps:** More data = better learning (reduces overfitting). Works well for image classification / object recognition, and also for speech.

**Limitation:** Augmentation requires **domain knowledge** about which transformations preserve the label.

## 14 Parameter Sharing and Tying

- **Parameter Sharing** (CNNs): Same filter applied at different positions of the image. Drastically reduces number of parameters.
- **Parameter Tying** (AEs): Encoder and decoder weights tied:  $W^* = W^\top$ . Reduces parameters from  $2d \times n$  to  $d \times n$ .

Both relate to model complexity via Stein's Lemma: fewer effective parameters  $\Rightarrow$  less sensitivity to data changes.

## 15 Adding Noise to Inputs

**Claim:** Adding Gaussian noise to inputs is equivalent to L2 regularisation.

Let  $\tilde{x}_i = x_i + \varepsilon_i$ ,  $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ ,  $\hat{y} = w^\top x$ :

$$\mathbb{E}[(\tilde{y} - y)^2] = \mathbb{E}[(\hat{y} - y)^2] + \underbrace{2\mathbb{E}\left[(\hat{y} - y) \sum_i w_i \varepsilon_i\right]}_{=0 \text{ (independence)}} + \mathbb{E}\left[\left(\sum_i w_i \varepsilon_i\right)^2\right]$$

Since  $\varepsilon_i$  are i.i.d. with  $\mathbb{E}[\varepsilon_i^2] = \sigma^2$ ,  $\mathbb{E}[\varepsilon_i \varepsilon_j] = 0$  for  $i \neq j$ :

$$\boxed{\mathbb{E}[(\tilde{y} - y)^2] = \mathbb{E}[(\hat{y} - y)^2] + \sigma^2 \|w\|^2}$$

This is exactly L2 regularisation with  $\lambda = \sigma^2$ .

## 16 Adding Noise to Outputs (Label Smoothing)

Instead of hard targets  $p = [0, \dots, 1, \dots, 0]$ , use **soft targets**:

$$p = \left[\frac{\varepsilon}{K}, \dots, 1 - \varepsilon, \dots, \frac{\varepsilon}{K}\right]$$

This prevents the model from driving training error to exactly zero and acts as regularisation by preventing **overconfidence**.

## 17 Early Stopping

**Algorithm 1** Early Stopping

- 
- 1: Track the validation error during training
  - 2: Have a patience parameter  $P$  (e.g.  $P = 5$ )
  - 3: Store the model at every epoch
  - 4: **if** at step  $k$ , no improvement for previous  $P$  steps **then**
  - 5:     **Stop training; Return** model from step  $k - P$
  - 6: **end if**
- 

**Early Stopping  $\approx$  L2 Regularisation**

Starting from  $w_0 = 0$ , using  $H = Q\Lambda Q^\top$  and the quadratic approximation, after  $T$  steps:

$$w_T = Q(I - (I - \eta\Lambda)^T)Q^\top w^*$$

Compare with L2 optimum:  $\tilde{w} = Q(\Lambda + \alpha I)^{-1}\Lambda Q^\top w^*$ .

They match when  $(I - \eta\Lambda)^T = (\Lambda + \alpha I)^{-1}\alpha$ .

**Key Insight**

Early stopping is the **most widely used** form of regularisation. Large  $\lambda_i$  (important directions):  $(1 - \eta\lambda_i)^T \approx 0 \Rightarrow$  direction fully updated. Small  $\lambda_i$ :  $(1 - \eta\lambda_i)^T \approx 1 \Rightarrow$  direction stays near zero. This exactly mirrors what L2 regularisation does.

## 18 Ensemble Methods and Bagging

---

**Bagging:** From a given dataset, construct multiple training sets by sampling with replacement:  $\mathcal{T}_1, \dots, \mathcal{T}_K$ .

### 18.1 Bagging MSE Analysis

Consider  $K$  models, each making error  $\varepsilon_i$  with  $\mathbb{E}[\varepsilon_i^2] = V$  and  $\mathbb{E}[\varepsilon_i \varepsilon_j] = C$  ( $i \neq j$ ):

$$\text{MSE}_{\text{ensemble}} = \frac{1}{K^2}[KV + K(K-1)C] = \frac{V}{K} + \frac{K-1}{K}C$$

**Bagging MSE**

$$\text{MSE}_{\text{ensemble}} = \frac{V}{K} + \frac{K-1}{K}C$$

- If  $C \approx 0$  (independent errors):  $\text{MSE} \approx V/K$  (reduces by factor  $K$ )
- If  $C = V$  (perfectly correlated):  $\text{MSE} = V$  (no improvement)

## 19 Dropout

---

### 19.1 The Dropout Mechanism

For each neuron  $j$ :  $\tilde{h}_j = r_j \cdot h_j$ ,  $r_j \sim \text{Bernoulli}(p)$ .

Each node is retained with probability  $p = 0.5$  (hidden) or  $p = 0.8$  (input). With  $n$  nodes, there are  $2^n$  possible thinned networks. Two tricks make this tractable:

1. **Share weights** across all  $2^n$  networks
2. **Sample a different network** for each training instance

## 19.2 Why Dropout Works: Preventing Co-Adaptation

### Key Insight

**Preventing Co-Adaptation:** Dropout prevents hidden units from relying too much on each other. Since other units may be dropped at any time, each hidden unit must learn to be **more robust**. This forces the model to learn genuinely general features.

## 19.3 Test Time: Weight Scaling

At test time, use the **full network** and scale each node's output by  $p$ :

$$w_{\text{test}} = p \cdot w_{\text{train}}$$

This approximates averaging over all  $2^n$  thinned networks.

## 20 Discrete Cosine Transform (DCT) and JPEG Compression

### 20.1 2D DCT Formula

The 2D DCT for an  $N \times N$  image  $f(x, y)$ :

$$F(u, v) = \frac{2}{N} C(u) C(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \cos\left(\frac{(2x+1)u\pi}{2N}\right) \cos\left(\frac{(2y+1)v\pi}{2N}\right)$$

where  $C(k) = 1/\sqrt{2}$  if  $k = 0$ , else 1.

### Note

Both DCT/JPEG and autoencoders aim for compact representations:

- **DCT:** Fixed cosine basis (no learning)
- **AE:** Learned basis (data-adaptive)

## 20.2 JPEG Compression Pipeline

1. **Colour Space Conversion** ( $\text{RGB} \rightarrow \text{YCbCr}$ ):  $Y$  = luminance,  $C_b/C_r$  = chrominance. Reversible.
2. **Chrominance Downsampling:** Each  $2 \times 2$  block of  $C_b, C_r \rightarrow$  one average value. Storage:  $3 \rightarrow 1.5$  values/pixel ( $2\times$  compression).
3. **Break into  $8 \times 8$  Blocks.**



4. **2D DCT of Each Block:** Concentrates energy in low-frequency coefficients (top-left).
5. **Quantisation:**  $\text{round}(\text{DCT coeff}/\text{quantisation table entry})$ . High-frequency coefficients round to 0.
6. **Run-Length + Huffman Encoding:** Read in zigzag order, encode runs of zeros.

**Why JPEG works:** Human eyes (100M rods for brightness, 6M cones for colour) are good at seeing edges but not high-frequency colour differences (individual grass blades, leaf clusters). JPEG removes exactly what our eyes cannot perceive.

**Impact:** 86% of all internet images are JPEGs. Video compression (H.264/AVC) uses similar principles.

## 21 Complete Summary

### 21.1 Key Equations at a Glance

**Autoencoder:**

$$\mathbf{h} = g(W\mathbf{x} + \mathbf{b}), \quad \hat{\mathbf{x}} = f(W^*\mathbf{h} + \mathbf{c}), \quad \mathcal{L} = \frac{1}{m} \left\| X - \hat{X} \right\|_F^2$$

**PCA–AE Equivalence:**

$$W = V_{:, \leq k}^\top, \quad H = X V_{:, \leq k}, \quad X = U \Sigma V^\top$$

**Bias–Variance Decomposition:**

$$\mathbb{E} \left[ (y - \hat{f}(x))^2 \right] = (\text{Bias})^2 + \text{Var} + \sigma^2$$

**L2 regularised solution:**

$$\tilde{w} = Q D Q^\top w^*, \quad D_{ii} = \frac{\lambda_i}{\lambda_i + \alpha}$$

**Bagging MSE:**

$$\text{MSE}_{\text{ensemble}} = \frac{V}{K} + \frac{K-1}{K} C \xrightarrow{C=0} \frac{V}{K} \xrightarrow{C=V} V$$

**Early stopping  $\approx$  L2:**

$$w_T = Q(I - (I - \eta\Lambda)^T)Q^\top w^* \approx \tilde{w} \quad \text{when} \quad (I - \eta\Lambda)^T = (\Lambda + \alpha I)^{-1}\alpha$$

**Dropout at test time:**  $w_{\text{test}} = p \cdot w_{\text{train}}$

## 21.2 Regularisation Methods Summary

| Method            | Mechanism                                                                                           |
|-------------------|-----------------------------------------------------------------------------------------------------|
| L2 / Weight Decay | $\lambda \ \theta\ ^2$ ; shrink weights; spectral scaling $D_{ii} = \lambda_i/(\lambda_i + \alpha)$ |
| Weight Tying      | $W^* = W^\top$ ; halves parameters                                                                  |
| Denoising         | Corrupt input $\tilde{x} \sim P(\tilde{x} x)$ ; reconstruct original                                |
| Sparse (KL)       | Enforce $\hat{\rho}_\ell = \rho$ ; neurons inactive most of time                                    |
| Contractive       | Penalise $\ J_f(\mathbf{h})\ _F^2$ ; reduce sensitivity to input                                    |
| Dataset Aug.      | Label-preserving transforms; more data                                                              |
| Param. Sharing    | Same weights at multiple places (CNNs)                                                              |
| Noise to inputs   | Equivalent to L2 ( $\lambda = \sigma^2$ )                                                           |
| Label smoothing   | Soft targets; prevent overconfidence                                                                |
| Early stopping    | Stop before overfitting; equivalent to L2                                                           |
| Ensemble/Bagging  | Average independent models; $\text{MSE} = V/K$ (if independent)                                     |
| Dropout           | Random thinned networks; $2^n$ models, shared weights                                               |

*“Regularisation: any method that reduces test error at the expense of training error.”*

## A Appendix A: Noise Inputs = L2 Regularisation (Full Derivation)

Let  $\tilde{x}_i = x_i + \varepsilon_i$ ,  $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ ,  $\hat{y} = w^\top x$ ,  $\tilde{y} = w^\top \tilde{x}_i = \hat{y} + \sum_i w_i \varepsilon_i$ .

$$\mathbb{E}[(\tilde{y} - y)^2] = \mathbb{E}[(\hat{y} - y)^2] + 0 + \sigma^2 \|w\|^2 = \mathbb{E}[(\hat{y} - y)^2] + \sigma^2 \|w\|^2$$

(Cross term vanishes by independence; noise term uses  $\mathbb{E}[\varepsilon_i^2] = \sigma^2$ ,  $\mathbb{E}[\varepsilon_i \varepsilon_j] = 0$ .) This is exactly L2 regularisation with  $\lambda = \sigma^2$ .  $\square$

## B Appendix B: Sparse AE – KL Divergence Gradient

$$\frac{\partial}{\partial \hat{\rho}_\ell} \text{KL}(\rho \| \hat{\rho}_\ell) = -\frac{\rho}{\hat{\rho}_\ell} + \frac{1 - \rho}{1 - \hat{\rho}_\ell}$$

$$\frac{\partial \tilde{\mathcal{L}}}{\partial w} = \frac{\partial \mathcal{L}}{\partial w} + \left( -\frac{\rho}{\hat{\rho}_\ell} + \frac{1 - \rho}{1 - \hat{\rho}_\ell} \right) \cdot \mathbf{x}_i (g'(w^\top \mathbf{x}_i + \mathbf{b}))^\top$$

## C Appendix C: Contractive AE – Jacobian Norm for Sigmoid Units

---

For  $h_\ell = \sigma(w_\ell^\top \mathbf{x} + b_\ell)$ :

$$\frac{\partial h_\ell}{\partial x_j} = h_\ell(1 - h_\ell) \cdot w_{\ell j} \implies \|J_f(\mathbf{h})\|_F^2 = \sum_{\ell=1}^k (h_\ell(1 - h_\ell))^2 \|w_\ell\|^2$$

Combined objective:

$$\min_{\theta} \mathcal{L}(\theta) + \lambda \sum_{\ell} (h_\ell(1 - h_\ell))^2 \|w_\ell\|^2$$

## D Appendix D: Early Stopping – Full L2 Correspondence

---

Starting from  $w_0 = 0$ , with  $H = Q\Lambda Q^\top$ :

$$w_{t+1} \approx (I - \eta H)w_t + \eta H w^* \implies w_T = Q(I - (I - \eta\Lambda)^T)Q^\top w^*$$

Scaling factor for direction  $i$  at step  $T$ :  $1 - (1 - \eta\lambda_i)^T$ .

L2 optimum scaling factor:  $\lambda_i/(\lambda_i + \alpha)$ . They match when  $1 - (1 - \eta\lambda_i)^T = \lambda_i/(\lambda_i + \alpha)$ .

Large  $\lambda_i$ : factor  $\approx 1$  (fully updated). Small  $\lambda_i$ : factor  $\approx 0$  (stays near zero). This exactly mirrors L2 regularisation.  $\square$

## E Appendix E: Bagging MSE – Full Derivation

---

$K$  models, errors  $\varepsilon_i$  with  $\mathbb{E}[\varepsilon_i^2] = V$ ,  $\mathbb{E}[\varepsilon_i \varepsilon_j] = C$  ( $i \neq j$ ):

$$\text{MSE} = \mathbb{E}\left[\left(\frac{1}{K} \sum_i \varepsilon_i\right)^2\right] = \frac{1}{K^2}[KV + K(K-1)C] = \frac{V}{K} + \frac{K-1}{K}C$$

When  $C = 0$ :  $\text{MSE} = V/K$ . When  $C = V$ :  $\text{MSE} = V$ .